

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ, СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**КЛИЕНТСКА БИБЛИОТЕКА ЗА ДОСТЪП ДО
БАЗИ ДАННИ ЧРЕЗ ПРЯКА РЕАЛИЗАЦИЯ НА
МРЕЖОВИТЕ ПРОТОКОЛИ НА POSTGRESQL
И MYSQL**

КУРСОВ ПРОЕКТ

по дисциплина „Надграждащи технологии за бази данни“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р инж. Надежда Кучмова

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Стандартът SQL и мястото на клиентския интерфейс в него	3
1.2. Технологии за достъп, изградени върху стандарта	3
1.3. Мрежовите протоколи на PostgreSQL и MySQL	4
1.4. Изводи от прегледа	4
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Разглеждани алтернативи	5
2.2. Критерии за сравнение	5
2.3. Обосновка на избора	6
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	7
3.1. Слоеве и отговорности	7
3.2. Протоколни кодеци	7
3.3. Декодиране без копие	7
3.4. Пул от връзки и кеш на подготвените заявления	8
3.5. Слой за типово изображение	8
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	9
4.1. Използвани средства	9
4.2. Структура на изходния текст	9
4.3. Ключови места в реализацията	9
4.4. Обработка на грешките	10
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	11
5.1. Какво се проверява	11
5.2. Еталони и натоварване	11
5.3. Среда и възпроизводимост	11
5.4. Показатели	12
5.5. Условия за валидност	12

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	13
6.1. Проверка на коректността	13
6.2. Закъснителност и пропускливост	13
6.3. Обръщания по мрежата и потребена памет	13
6.4. Анализ	14
ЗАКЛЮЧЕНИЕ	15
ЛИТЕРАТУРА	16
ПРИЛОЖЕНИЯ	18

УВОД

Достъпът до релационна база данни от програма на С или С++ почти винаги минава през чужд слой. Приложението извиква клиентска библиотека на доставчика, тя от своя страна говори с процеса на сървъра по мрежов протокол, а между двете нерядко стои и диспечер на драйвери като ODBC. Всеки слой има своя цена: собствен модел на паметта, собствено разбиране за типовете, собствено копие на резултатните редове и собствен, обикновено синхронен, модел на изчакване. Приложението плаща тази цена, без да я вижда, защото слоят е даденост.

Настоящият курсов проект разглежда алтернативата. Мрежовите протоколи на PostgreSQL и на MySQL са публично специфицирани [1, 2], което означава, че клиентът може да бъде написан срещу самата спецификация, а не срещу чужда библиотека. Този избор премахва междинните слоеве и прави видими решения, които иначе са скрити: къде се копира буфер, кога се сериализира стойност, колко обръщания до сървъра струва едно подготвено заявление. Същият избор носи и очевидни рискове, които проектът е длъжен да отчете честно, а не да премълчи.

Целта на проекта е да се проектира и реализира клиентска библиотека на С++20, която реализира пряко мрежовите протоколи на PostgreSQL и MySQL, и да се измери поведението ѝ спрямо доставчиковите клиентски библиотеки върху едно и също натоварване.

За постигането на целта са формулирани следните **задачи**:

1. преглед на стандарта SQL и на изградените върху него технологии за достъп, както и на спецификациите на двата разглеждани мрежови протокола;
2. анализ на възможните решения, слоест диспечер на драйвери, доставчикова клиентска библиотека или пряка реализация на протокола, и обосновка на избора;
3. проектиране на слоеста архитектура с ясно отделени транспорт, протоколни кодеци, декодер на редове, пул от връзки и слой за типово изображение;
4. програмна реализация на архитектурата за двата протокола;
5. изграждане на методика за измерване и на възпроизводима среда за нея;
6. експериментално сравнение с `libpq` и `libmysqlclient` и анализ на получените резултати.

Обхватът на проекта е клиентската страна на комуникацията: установяване на връзка, удостоверяване, изпълнение на прости и на подготвени заявления, декодиране на резултатни редове и управление на пула от връзки. Извън обхвата остават администрирането на сървъра, репликацията, разпределените транзакции и всяка форма на съхранение от страна на клиента.

Структура на изложението. Раздел I представя предметната област и прегледа на литературата. Раздел II излага разгледаните алтернативи и обосновава избора. Раздел III описва проектирането, а Раздел IV програмната реализация. Раздел V дефинира методиката за измерване, а Раздел VI представя резултатите и техния анализ. Заключението обобщава постигнатото и посочва посоките за продължение.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

1.1. Стандартът SQL и мястото на клиентския интерфейс в него

Стандартът ISO/IEC 9075 описва не само езика за заявки, но и начина, по който външна програма стига до данните. Част 1 задава общата рамка и понятията, с които останалите части си служат [3]. Част 3 определя *интерфейса на ниво извикване* (Call-Level Interface, CLI), тоест процедурен интерфейс, чрез който приложение на общоцелеви език изпраща заявления и получава резултати, без тези заявления да са вградени в кода при компилация [4]. Именно тази част е шаблонът, върху който стъпват практическите технологии за достъп.

Разграничението, което стандартът прави, е съществено за целия проект. Стандартът нормира *интерфейса*, който приложението вижда, но не нормира *протокола*, по който клиентът и сървърът си говорят по мрежата. Протоколът е дело на всеки доставчик поотделно. Следователно преносимостта, която CLI обещава, се постига винаги чрез междинен слой, който превежда стандартните извиквания към нестандартния протокол на конкретния сървър.

1.2. Технологии за достъп, изградени върху стандарта

ODBC е най-разпространената реализация на идеята за CLI. Приложението се свързва с *диспечер на драйвери*, който зарежда драйвер за конкретния източник на данни и пренасочва извикванията към него [5]. Върху този модел исторически се надграждат OLE DB, ADO и по-късно ADO.NET, които добавят обектен модел, работа с несвързани множества от редове и по-богата типова система [6]. Общото между всички тях е броят на границите, през които минава един резултатен ред, преди да стигне до приложението.

Отделно от релационния път стоят спецификациите за слабо структурирани данни и за директорийни услуги. XML остава нормативната основа за обмен на документно структурирани данни между разнородни системи [7], а LDAP описва достъпа до йерархични директорийни хранилища с протокол, който по устройство е близък до разглежданите тук: двоичен обмен на съобщения с ясно определени етикети и дължини, върху постоянна връзка [8]. Сравнението с LDAP е полезно, защото показва, че прякото

говорене по протокол е обичайна практика извън релационния свят.

1.3. Мрежовите протоколи на PostgreSQL и MySQL

Протоколът на PostgreSQL е съобщително ориентиран: всяко съобщение носи еднобайтов етикет, дължина и тяло, а фазите на работа са ясно отделени, стартиране и удостоверяване, прост цикъл на заявка, разширен цикъл с Parse/Bind/Execute за подготвени заявления [1]. Протоколът на MySQL е организиран около пакети с последователен номер и разчита в по-голяма степен на позиционно кодиране и на цели числа с променлива дължина [2]. Двата протокола решават една и съща задача с различни средства, което ги прави удобна двойка за общ клиентски слой.

Референтните клиентски библиотеки на двата доставчика са добре документирани и служат за еталон в тази работа. Документацията на libpq описва подробно както интерфейса, така и семантиката на асинхронния режим [9]. По-общата литература за системи за обработка на транзакции обяснява защо клиентският слой не е неутрален към производителността: границите на транзакциите, изчакванията и броят обръщания по мрежата определят наблюдаваната закъснителност повече от скоростта на самия сървър [10, 11].

1.4. Изводи от прегледа

Прегледът очертава три нива, на които може да се стъпи: стандартизиран интерфейс с диспечер на драйвери, доставчикова клиентска библиотека и самата спецификация на протокола. Всяко следващо ниво надолу премахва слой и абстракция, но добавя отговорност за коректност. **[TODO: да се добавят два или три съпоставими академични или индустриални източника за измервана цена на слоевете при достъп до бази данни, ако такива бъдат намерени; ако не бъдат, това да се заяви изрично тук]**

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Разглеждани алтернативи

Разгледани са три решения на една и съща задача, а именно да се изпълни заявление от програма на C++ и да се получат резултатните редове в типизиран вид.

Алтернатива А, слоест диспечер на драйвери. Приложението извиква ODBC, а диспечерът зарежда драйвер за конкретния сървър [5]. Предимството е една входна точка за много източници на данни, включително такива, за които никой не би написал клиент сам. Недостатъкът е, че всеки резултатен ред минава през договорения от стандарта тип и през буфер, зададен от приложението предварително, а асинхронната работа не е част от модела.

Алтернатива Б, доставчикова клиентска библиотека. Приложението се свързва пряко с `libpq` или с `libmysqlclient`. Отпада един слой, а семантиката е точно тази на сървъра. Остават обаче два въпроса: библиотеките са на C и налагат собствено управление на паметта на границата, а асинхронните им режими се различават помежду си достатъчно, за да не се обединяват просто [9].

Алтернатива В, пряка реализация на мрежовия протокол. Приложението кодира и декодира съобщенията само, срещу публикуваната спецификация [1, 2]. Отпадат всички междинни слоеве, буферът за получаване се притежава от самата библиотека и декодирането може да работи върху него без копие. Цената е, че коректността вече е отговорност на проекта, включително частите на протокола, които се срещат рядко и затова се откриват късно.

2.2. Критерии за сравнение

Сравнението се води по критерии, всеки от които е проверим, а не въпрос на вкус: брой слоеве, през които минава един ред; брой копия на данните между сокета и типизираната стойност; наличие на естествен асинхронен модел; възможност за кеширане на подготвени заявления от страна на клиента; обем на кода, който проектът поема да поддържа; чувствителност към промени във версията на сървъра.

Таблица 1. Съпоставка на разгледаните алтернативи по приетите критерии

Критерий	A. ODBC	Б. Доставчикова	В. Пряк протокол
Слоеве до реда	[TODO: ?]	[TODO: ?]	[TODO: ?]
Копия на данните	[TODO: ?]	[TODO: ?]	[TODO: ?]
Асинхронен модел	[TODO: ?]	[TODO: ?]	[TODO: ?]
Кеш на заявленията	[TODO: ?]	[TODO: ?]	[TODO: ?]
Поддържан код	[TODO: ?]	[TODO: ?]	[TODO: ?]

Съпоставката по тези критерии (Табл. 1) се попълва след като реализацията и измерванията приключат, за да не съдържа предположения.

2.3. Обосновка на избора

Проектът избира алтернатива В. Основанието не е производителност сама по себе си, а това, че само тя позволява цената на достъпа да бъде наблюдавана. При А и Б броят копия и обръщания е скрит зад чужд код и може да се оцени само косвено. При В всяко копие е ред в собствения изходен текст и следователно може да бъде премахнато или защитено с аргумент. Дисциплината изисква и сравнение между слоестия модел и прякото говорене по протокол, което тази постановка прави естествено.

Изборът се приема заедно с цената му. Проектът поема поддръжката на две независими кодиращи схеми, поема риска от разминаване с бъдещи версии на сървърите и се отказва от преносимостта към източници на данни извън двата разглеждани. Тези ограничения са записани изрично, за да бъдат отчетени в заключението, а не преоткрити там.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Слоеви и отговорности

Системата е разделена на пет слоя с еднопосочна зависимост, отдолу нагоре. Най-отдолу стои *транспортът*, който притежава сокета и буферите и не знае нищо за съдържанието на байтовете. Над него стои *протоколният кодек*, по един за PostgreSQL и за MySQL, който превръща поток от байтове в съобщения и обратно. Следва *декодерът на редове*, който превръща тялото на едно съобщение с данни в изгледи към стойности, без да ги копира. Над него е *пултът от връзки*, който управлява живота на връзките и кеша от подготвени заявления. Най-отгоре стои *слоят за типово изображение*, който свързва стойностите от протокола с типовете на C++.

Границата между всеки два слоя е тесен договор, а не общ обект със състояние. Целта е всеки слой да се тества самостоятелно: кодекът да се захранва със записан байтов поток, декодерът с готово тяло на съобщение, пултът с фиктивен транспорт. Това разделение е и условието за смисленото сравнение с доставчиците библиотеки, защото позволява цената да се отнесе към конкретен слой.

3.2. Протоколни кодекси

Кодекът на PostgreSQL следва съобщителния модел на протокола: етикет, дължина, тяло, и двата цикъла на изпълнение, простия и разширения [1]. Кодекът на MySQL следва пакетния модел с номер на последователност и цели числа с променлива дължина [2]. Двата кодекса нямат общ базов клас. Общото между тях е изразено като изисквания към типа, тоест като *сепарат*, което позволява останалите слоеве да са шаблонни и да не плащат за динамично изпращане.

[TODO: фигура: диаграма на слоевете и на потока от данни между сокета и типизираната стойност; да се начертае с TikZ и да се озаглави по правилото за фигури]

3.3. Декодиране без копие

Резултатният ред пристига като част от получаващия буфер, притежаван от транспорта. Декодерът не изгражда нови обекти за стойностите, а издава изгледи от вида

`std::span<const std::byte>` и `std::string_view` към този буфер. Копие се прави само когато приложението изрично поиска притежание на стойността. Условието за коректност е ясно: изгледите са валидни, докато редът е жив, и това ограничение е част от публичния договор, а не подразбираща се подробност.

Тази схема прехвърля отговорност върху приложението и затова е рискова. Проектът я приема, но я подсигурава с проверки при активирани диагностики и с тестове, които целенасочено използват изглед към вече невалиден ред, за да се убедят, че диагностиката го улавя. **[TODO: да се опише точният механизъм за проверка след като бъде избран]**

3.4. Пул от връзки и кеш на подготвените заявления

Пулът е асинхронен. Заявка за връзка, която не може да бъде удовлетворена веднага, спира изпълнението на извикващата съпрограма, вместо да блокира нишка. Всяка връзка носи собствен кеш на подготвените заявления, защото подготвеното заявление в двата разглеждани протокола е свързано със сесията, а не с базата. Кешът е с ограничен размер и с политика на изместване. **[TODO: избор на политика и на подразбиращ се размер, след като измерванията покажат чувствителността към тях]**

3.5. Слой за типово изображение

Слоят превръща стойност от протокола в типизирана стойност на C++ и обратно, като съответствието се задава декларативно и се проверява при компилация. Целта е несъответствие между схемата и заявената в кода структура да бъде грешка при компилация или ясна грешка при изпълнение, а не мълчаливо погрешно четене на байтове. **[TODO: таблица със съответствието между типовете на PostgreSQL, типовете на MySQL и типовете на C++, включително случаите без точно съответствие]**

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Използвани средства

Реализацията е на C++20 [12]. От стандарта се използват три възможности, всяка с конкретно основание. Съпрограмите носят асинхронния модел, без да налагат обратни извиквания и без да раздробяват логиката на протокола между функции. Изискванията към типовете, тоест concept, изразяват общото между двата кодека без виртуални функции. `std::span` и `std::string_view` са носителите на декодирането без копие.

Сглобяването е с CMake, тестовите са на GoogleTest, а измерванията се извършват от отделна програма, а не в тестовите, за да не се смесват двата вида проверка. Интеграционните тестове изискват работещи сървъри и затова средата се вдига с Docker Compose, което прави версиите на сървърите част от конфигурацията и следователно записани. **[TODO: версии на PostgreSQL и MySQL, използвани при измерванията]**

4.2. Структура на изходния текст

Изходният текст е разделен по слоевете от Раздел III. Публичните заглавни файлове са в `include/`, реализацията в `src/`, тестовите в `tests/`, измервателната програма в `benchmarks/`. Двата протокола са в отделни поддървета, които не се включват взаимно. Общото между тях е само в слоя на транспорта и в изискванията към типовете.

[TODO: дърво на директориите след като реализацията стабилизира структурата си]

4.3. Ключови места в реализацията

Три места носят по-голямата част от сложността и затова са коментирани отделно. Първото е разчитането на непълно съобщение: транспортът може да върне произволна част от пакет, така че кодекът трябва да е способен да спре и да продължи, без да губи състояние. Второто е грешката от страна на сървъра в средата на резултатен поток, която задължава клиента да изчерпи протоколния обмен до синхронизиращата точка, преди връзката да бъде върната в пула. Третото е анулирането на съпрограма, докато операция по мрежата е в ход.

[TODO: линтинг: обработка на непълно съобщение в кодека на PostgreSQL]

[TODO: листинг: заемане и връщане на връзка от асинхронния пул]

4.4. Обработка на грешките

Грешките се разделят на три вида: протоколна грешка, тоест байтов поток, който не отговаря на спецификацията; грешка, съобщена от сървъра по правилата на протокола; и грешка на транспорта. Първите две се различават съществено, защото само втората оставя връзката годна за по-нататъшна употреба. Връзка, която е получила протоколна грешка, се затваря, а не се връща в пула. **[TODO: да се опише представянето на грешките в интерфейса, след като изборът между изключения и тип със стойност или грешка бъде направен и обоснован]**

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

5.1. Какво се проверява

Проверката има две отделни цели, които не бива да се смесват. Първата е коректност: реализацията трябва да връща същите резултати като доставчиковата библиотека за едно и също заявление върху една и съща база. Втората е цена: закъснителност, пропускливост и потребена памет при едно и също натоварване. Коректността е предусловие. Измерване върху реализация, която дава различни резултати, няма смисъл.

5.2. Еталони и натоварване

Еталоните са `libpq` за PostgreSQL и `libmysqlclient` за MySQL. Сравнението се прави върху едно и също натоварване, изпълнено от една и съща програма, в която се сменя само клиентският слой. Натоварването включва прости заявления, подготвени заявления с параметри, четене на голям резултат и работа при няколко едновременни връзки. **[TODO: точно описание на схемата, обема на данните и списъка от заявления]**

5.3. Среда и възпроизводимост

Средата се вдига с Docker Compose, за да са фиксирани версиите на сървърите и началното състояние на базите. Записват се: версия на операционната система, версия на компилатора и режим на сглобяване, версии на сървърите, версии на еталонните библиотеки, параметри на машината. **[TODO: конфигурация на машината и версии на всички изброени компоненти]**

Всяко измерване се повтаря няколко пъти, като първите повторения се отхвърлят като загряващи. Отчита се разсейването, а не само средната стойност, защото средна стойност без разсейване не позволява да се съди дали разликата между два клиента изобщо е значима. **[TODO: брой повторения, брой загряващи повторения и правило за отхвърляне на измерване]**

5.4. Показатели

Измерват се четири показателя: закъснителност на едно заявление по проценти на разпределението, пропускливост при фиксиран брой едновременни връзки, върхова потребена памет за целия процес и брой обръщания по мрежата за едно логическо действие. Последният показател не е времеви и се получава чрез броене в самия клиент, което го прави независим от машината и затова най-удобен за сравнение между реализации.

5.5. Условия за валидност

Измерването се приема за валидно само ако са изпълнени следните условия: и двата клиента изпълняват едно и също заявление върху една и съща база; сървърът е един и същ процес за двете измервания; резултатите на двата клиента съвпадат байт по байт след типовото изображение; загорящите повторения са отхвърлени; и разсейването е отчетено. Измерване, което не отговаря на някое от условията, се записва като невалидно и не участва в изводите. **[TODO: да се опише как се проверява всяко от условията автоматично]**

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

6.1. Проверка на коректността

Преди всяко сравнение по цена се докладва резултатът от проверката за съвпадение на резултатите между разработената библиотека и еталонната. Докладват се броят изпълнени случаи, броят съвпадения и описанието на всяко разминаване. **[TODO: резултат от проверката за коректност; таблицата се попълва след като интеграционните тестове започнат да се изпълняват срещу двата сървъра]**

6.2. Закъснителност и пропускливост

Таблица 2. Закъснителност на едно заявление, PostgreSQL, сравнение с libpq

Вид заявление	Медиана	95-и процентил	Разсейване
Просто заявление	[TODO: ?]	[TODO: ?]	[TODO: ?]
Подготвено заявление	[TODO: ?]	[TODO: ?]	[TODO: ?]
Голям резултат	[TODO: ?]	[TODO: ?]	[TODO: ?]

Резултатите за PostgreSQL (Табл. 2) се придружават от аналогична таблица за MySQL и от коментар за всяко разминаване, което надхвърля измереното разсейване. **[TODO: таблица за MySQL със същата форма]**

6.3. Обръщания по мрежата и потребена памет

Таблица 3. Брой обръщания по мрежата и върхова потребена памет за едно логическо действие

Действие	Обръщания, еталон	Обръщания, проект	Памет
Свързване	[TODO: ?]	[TODO: ?]	[TODO: ?]
Първо изпълнение	[TODO: ?]	[TODO: ?]	[TODO: ?]
Повторно изпълнение	[TODO: ?]	[TODO: ?]	[TODO: ?]

Броят обръщания (Табл. 3) е показателят, при който кешът на подготвените заявления се очаква да има най-ясен ефект, защото повторното изпълнение спестява

фазата на подготовка. Дали ефектът се проявява и при какъв размер на кеша е въпрос към измерването, а не към проектирането. **[TODO: графика на закъснителността спрямо размера на кеша]**

6.4. Анализ

Анализът тълкува получените числа и назовава причината за всяко разминаване, като разграничава три възможни източника: разлика в броя копия, разлика в броя обръщания и разлика, дължаща се на самия сървър, тоест обща и за двата клиента. **[TODO: анализ, след като има какво да се анализира]**

[TODO: изрично да се отбележат хипотезите от Раздел II, които измерването е опровергало, ако има такива]

ЗАКЛЮЧЕНИЕ

Проектът разработва клиентска библиотека на C++20, която реализира пряко мрежовите протоколи на PostgreSQL и MySQL, с асинхронен пул от връзки, кеш на подготвените заявления и декодиране на редове без копие. Изборът да се говори по протокола, вместо да се използва слоест диспечер на драйвери, е направен съзнателно и е обоснован в Раздел II с това, че прави цената на достъпа наблюдаема.

Предимства на решението. Липсата на междинни слоеве премахва копия, които иначе са скрити. Асинхронният модел е естествен за C++20 и не изисква нишка на връзка. Типовото изображение се проверява при компилация. **[TODO: предимствата да се потвърдят или отрекат с числата от Раздел VI]**

Недостатъци и ограничения. Проектът поема поддръжката на две независими протоколни реализации и носи риска от разминаване при нови версии на сървърите. Обхватът на протоколите не е пълен и това е съзнателно ограничение, а не пропуск. Декодирането без копие прехвърля отговорност за живота на данните върху приложението. **[TODO: списък на частите от двата протокола, които реализацията не покрива]**

Инженерна трактовка на резултатите. **[TODO: извод от измерванията: къде прякото говорене по протокол дава измерима полза, къде разликата е в границите на разсейването и къде еталонната библиотека е по-добра]**

Посоки за по-нататъшна работа. Естествените продължения са три: разширяване на покритието на протоколите до частите, които днес остават извън обхвата; добавяне на трети протокол, което би проверило доколко общият слой наистина е общ; и слой за съвместимост с интерфейс от типа на CLI [4], който би позволил библиотеката да се използва от код, писан срещу стандартния интерфейс.

ЛИТЕРАТУРА

- [1] PostgreSQL Global Development Group, “Frontend/Backend Protocol.” PostgreSQL Documentation, <https://www.postgresql.org/docs/current/protocol.html>. **[TODO: дата на достъп]**.
- [2] Oracle Corporation, “MySQL Client/Server Protocol.” MySQL Internals Manual, https://dev.mysql.com/doc/dev/mysql-server/latest/PAGE_PROTOCOL.html. **[TODO: дата на достъп]**.
- [3] International Organization for Standardization, “Information technology, Database languages, SQL, Part 1: Framework (SQL/Framework),” Tech. Rep. ISO/IEC 9075-1:2023, ISO/IEC, 2023.
- [4] International Organization for Standardization, “Information technology, Database languages, SQL, Part 3: Call-Level Interface (SQL/CLI),” Tech. Rep. ISO/IEC 9075-3:2023, ISO/IEC, 2023.
- [5] Microsoft Corporation, “ODBC Programmer’s Reference.” Microsoft Learn, <https://learn.microsoft.com/en-us/sql/odbc/reference/odbc-programmer-s-reference>. **[TODO: дата на достъп]**.
- [6] Microsoft Corporation, “ADO.NET Overview.” Microsoft Learn, <https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/ado-net-overview>. **[TODO: дата на достъп]**.
- [7] T. Bray, J. Paoli, C. M. Sperberg-McQueen, E. Maler, and F. Yergeau, “Extensible Markup Language (XML) 1.0 (Fifth Edition).” W3C Recommendation, <https://www.w3.org/TR/xml/>, 2008. **[TODO: дата на достъп]**.
- [8] J. Sermersheim, “Lightweight Directory Access Protocol (LDAP): The Protocol,” Tech. Rep. RFC 4511, Internet Engineering Task Force, 2006.
- [9] PostgreSQL Global Development Group, “libpq: C Library.” PostgreSQL Documentation, <https://www.postgresql.org/docs/current/libpq.html>. **[TODO: дата на достъп]**.
- [10] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

- [11] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [12] International Organization for Standardization, “Programming languages, C++,” Tech. Rep. ISO/IEC 14882:2020, ISO/IEC, 2020.

ПРИЛОЖЕНИЯ

Приложение А. Схема на тестовата база

[TODO: изходен текст на схемата, използвана при измерванията, заедно с обема на данните във всяка таблица]

Приложение Б. Конфигурация на средата

[TODO: файлът на Docker Compose и версиите на сървърите, с които са направени измерванията]

Приложение В. Избрани части от изходния текст

[TODO: кодът на съобщенията за PostgreSQL]

[TODO: кодът на пакетите за MySQL]

[TODO: асинхронният пул от връзки]

Приложение Г. Пълни резултати от измерванията

[TODO: суровите резултати от всички повторения, от които са изведени таблиците в Раздел VI]